

Q.NO :9

Question description:

On the last day of the semester, Shahid's students were talking and playing very loudly to the delight of the end of the semester. The principal of the college severely reprimanded Shahid. But he decided to engage them in a different activity without getting angry at the students.

So Shahid gave his students to solve the task such that, Circular Queue using linked list and dequeue two elements from circular queue.

there is no memory waste while using Circular Queue, it is preferable than using a regular queue.

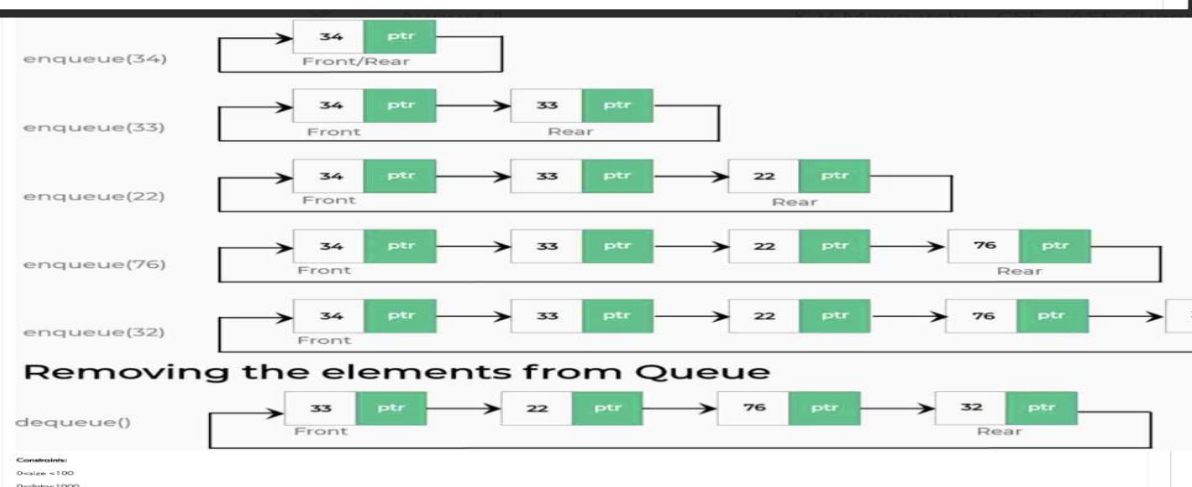
Because linked lists allow for dynamic memory allocation, they are simple to build.

Circular Queue implementation using linked list is identical to circular linked list except that circular Queue has two pointers front and back whereas circular linked list only has one pointer head.

Final Year students were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Function Description



Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

First line indicates the inserted elements in queue using linked list concept

second line indicates the one element dequeued from the queue using linked list concept.

Logical Test Cases

Test Case 1

INPUT (STDIN)

5
9 1 5 3 5

EXPECTED OUTPUT

9 1 5 3 5
1 5 3 5
5 3 5

Test Case 2

INPUT (STDIN)

8
1 8 6 4 9 5 3 5

EXPECTED OUTPUT

1 8 6 4 9 5 3 5
8 6 4 9 5 3 5
6 4 9 5 3 5

Mandatory Test Cases

Test Case 1

KEYWORD

void enqueue(int d)

Test Case 4

KEYWORD

struct node *f = NULL;

Test Case 7

KEYWORD

n->next = NULL;

Test Case 2

KEYWORD

struct node* next;

Test Case 5

KEYWORD

struct node *r = NULL;

Test Case 8

KEYWORD

struct node* t;

Test Case 3

KEYWORD

n = (struct node*)malloc(sizeof(struct node));

Test Case 6

KEYWORD

n->data = d;

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node* next;
};

struct node *f = NULL;
struct node *r = NULL;

void enqueue(int d)
{
    struct node *n;

    n = (struct node*)malloc(sizeof(struct node));

    n->data = d;
    n->next = NULL;

    if(f == NULL)
    {
        f = n;
        r = n;
        n->next = f;
    }
    else
    {
        n->next = f;
        r->next = n;
        r = n;
    }
}

void dequeue()
{
    struct node *t;

    if(f == NULL)
    {
        return;
    }

    t = f;

    if(f == r)
    {
        f = NULL;
        r = NULL;
    }
}
```

```
}
else
{
f = f->next;
r->next = f;
}

free(t);
}

void print()
{
struct node *t;

if(f == NULL)
{
return;
}

t = f;

do
{
printf("%d", t->data);

if(t->next != f)
printf(" ");

t = t->next;
} while(t != f);

printf("\n");
}

int main()
{
int n, data;

printf("Input value:\n");

scanf("%d", &n);

for(int i = 0; i < n; i++)
{
scanf("%d", &data);
enqueue(data);
}

printf("\nOutput:\n");
```

```
print();
dequeue();
print();

dequeue();
print();

return 0;
}
```

Output:

Test_case 1:

```
aadityajaiswal@Aadityas-MacBook-Air:~$
5
9 1 5 3 5
9 1 5 3 5
1 5 3 5
5 3 5
```

Test_case 2:

```
8
1 8 6 4 9 5 3 5
1 8 6 4 9 5 3 5
8 6 4 9 5 3 5
6 4 9 5 3 5
```